



TRAFFIC SIGN DETECTION USING YOLO

FINE-TUNING YOLO26N ON A 15-CLASS SELF-DRIVING CAR DATASET

MINI RESEARCH PROJECT – ZIRU XU

TLDR

- **Motivation**

Traffic sign recognition is a critical component of autonomous driving and driver assistance systems. Accurate and real-time detection helps vehicles understand road regulations, improve safety, and support navigation decisions.

- **Main Idea**

Fine-tuned a YOLO26 object detection model on the Self-Driving Cars Traffic Sign Dataset from Roboflow to automatically identify and localize traffic signs in road images. The model learns bounding boxes and sign categories directly from labeled training data.

- **Results**

The fine tuned model achieves strong overall detection: Precision 82.5% | Recall 72.8% | mAP@0.5 82.4% | mAP@0.5:0.95 72.0%, with recall as the main improvement area.

LITERATURE REVIEW

Evolution of Traffic Sign Detection



From Handcrafted Features to Real-Time AI for Autonomous Driving

2000s

Traditional Methods

Color Segmentation, HOG, SIFT, Shape Analysis



- ✓ Fast and simple
- ✗ Sensitive to lighting, occlusion and complex backgrounds

(Maldonado-Bascón et al., 2007)

2012+

CNN-Based Recognition

Deep CNNs learn discriminative features



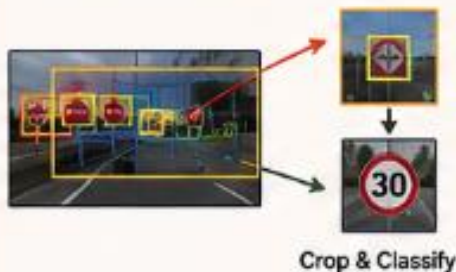
- ✓ Automatic feature learning
- ✓ Higher classification accuracy
- ✗ Primarily focused on classification

(Krizhevsky et al., 2012)

2015+

Faster R-CNN

Two-stage detector with Region Proposal Network



- ✓ High detection accuracy
- ✗ Computationally expensive slower inference

(Ren et al., 2015)

2016+

YOLO Family

Single-stage detector for real-time performance



- ✓ Real-time detection
- ✓ Good trade-off between speed and accuracy
- ✗ Challenges with very small objects

(Redmon et al., 2016)

This Project



YOLO26 for Traffic Sign Detection

Modern architecture for accurate and efficient detection

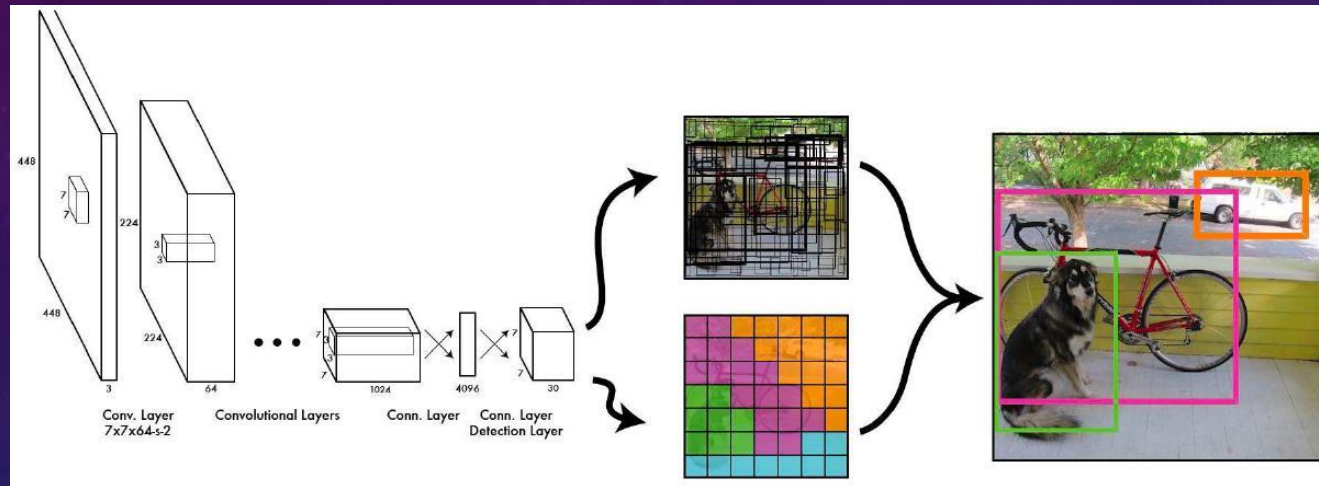


- ✓ Fast and end-to-end detection
- ✓ Accurate localization of multiple sign classes
- ✓ Suitable for real-world autonomous driving

(Our Work)

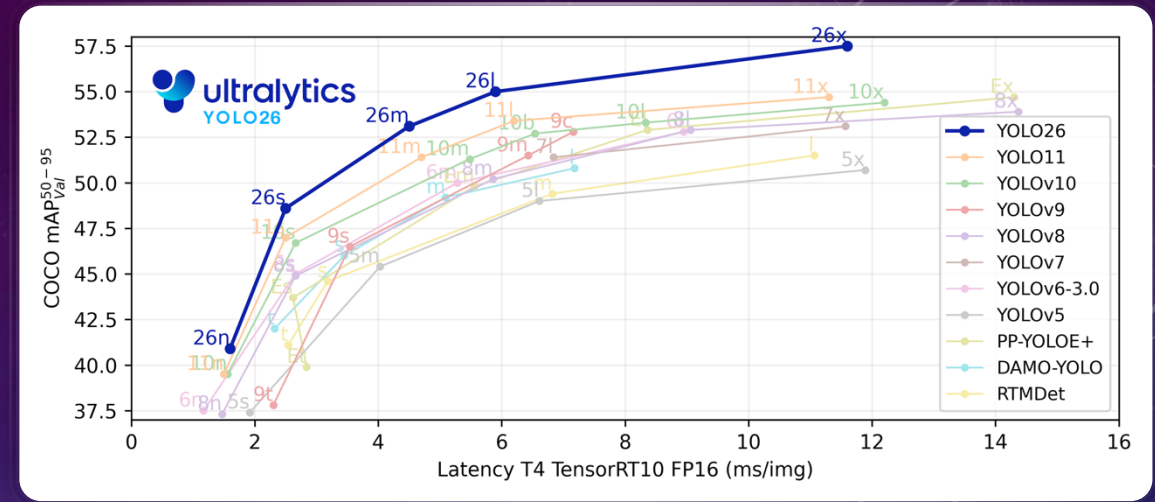


LITERATURE REVIEW - YOLO



- **Overall structure:** A single neural network that takes an image and directly outputs bounding boxes + class predictions
- **CNN feature extraction + detection head:** Uses a CNN backbone to extract features, then a detection head to predict objects
- **One-step detection:** No separate proposal or region stages → everything happens in one forward pass
- **Grid-based (original idea):** Image is divided into grids; each grid predicts objects in its region
- **Multi-task learning:** Jointly learns object location, confidence, and class in one model

LITERATURE REVIEW



- YOLOv1 (2016): First single-stage real-time object detector; fast but limited accuracy
- YOLOv3 (2018): Introduced multi-scale detection; became a widely used baseline
- YOLOv5 (2020): PyTorch-based implementation; major boost in usability and industry adoption
- YOLOv8 (2023): Anchor-free design; unified framework for detection, segmentation, and classification
- YOLOv9–YOLOv10 (2024–2025): Focus on efficiency, training stability, and low-latency inference YOLO
- 2026 / YOLO26: Next-gen direction emphasizing foundation-model style generalization and edge deployment

APPROACH - DATA

Source: Roboflow Self-Driving Cars dataset

Train Set: 3530 Images (71%)

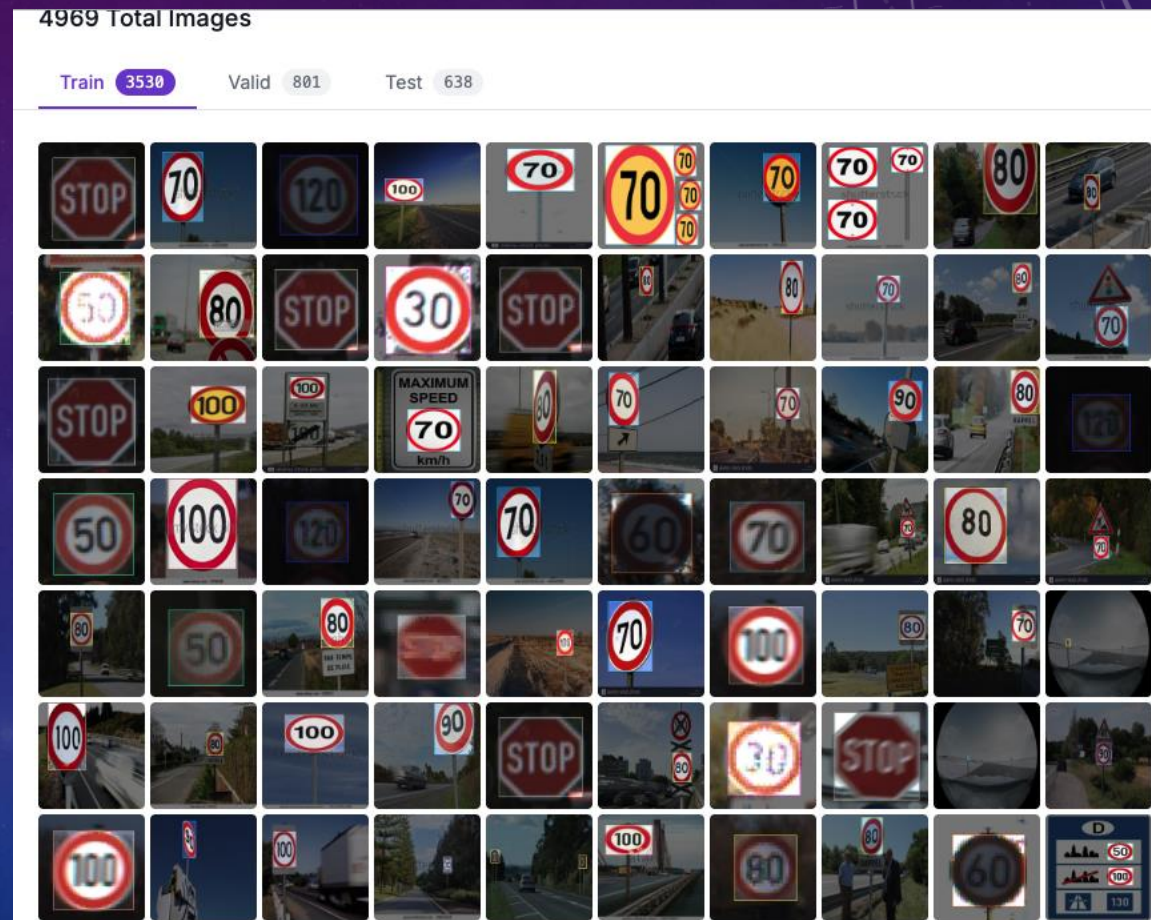
Valid Set: 801 Images (16%)

Test Set: 638 Images (13%)

15 Classes (Stop sign, Red Light, Green Light, Different Speed Limit)

Resize: Stretch to 416x416

All images are real car-perspective dashcam frames with YOLO-format bounding box labels (one .txt per image).



APPROACH – MODEL & FINETUNE

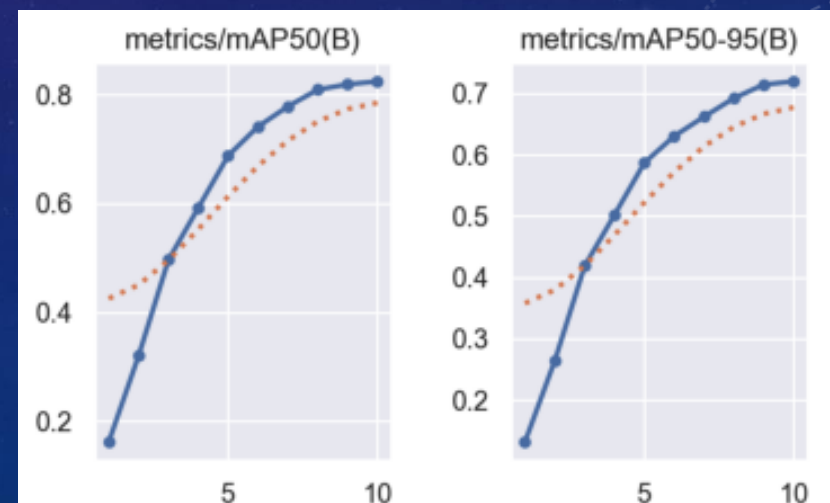
- **Model — YOLO26n**

- Nano (n) variant: lightweight + edge-friendly design
- 2.4M parameters | 5.4 GFLOPs | ~40.9 mAP (COCO baseline)

- **Fine-tuning Strategy**

- Initialized from COCO-pretrained weights (yolo26n.pt)
- Leverages 80-class general visual feature representations for transfer learning
- Fine-tuned on traffic sign dataset for domain-specific adaptation.

Parameter	Value	Meaning
epochs	10	Number of full passes over training data
batch	-1 (auto)	YOLO auto-selects batch size to fill ~60% RAM
optimizer	auto	YOLO selects SGD or AdamW based on model size
lr0	0.01	Initial learning rate
lrf	0.01	Final LR = $lr0 \times lrf$ (cosine decay)
momentum	0.937	SGD momentum
weight_decay	0.0005	L2 regularisation



APPROACH – EVALUATION

Evaluate on the 801-image validation split

- **Precision: 82.5%**
→ When the model predicts an object, it is correct most of the time (low false positives)
- **Recall: 72.8%**
→ The model detects most objects, but still misses ~27% (moderate false negatives)
- **mAP@0.5: 82.4%**
→ Strong detection performance with reasonable localization accuracy (lenient IoU threshold)
- **mAP@0.5:0.95: 72.0%**
→ Good overall performance under strict COCO evaluation (robust box accuracy)

DEMO & RESOURCES

- Youtube Demo: <https://www.youtube.com/watch?v=TWE2qykZR1M>
- GitHub Link: <http://github.com/zxu235/ai-mrp-spring-2026-Ziru-Xu>
- Future Work
 - **Extended training:** Run GPU training for 50+ epochs to achieve full model convergence and improve overall accuracy
 - **Robust data augmentation:** Introduce challenging conditions such as fog, night, rain, and motion blur to improve real-world robustness

REFERENCES

- Flores-Calero, M., Astudillo, C. A., Guevara, D., Maza, J., Lita, B. S., Defaz, B., Ante, J. S., Zabala-Blanco, D., & Armingol Moreno, J. M. (2024). *Traffic Sign Detection and Recognition Using YOLO Object Detection Algorithm: A Systematic Review*. Mathematics, 12(2), 297.
<https://doi.org/10.3390/math12020297>
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection*. Proceedings of CVPR.
- Jiang, L., Zhan, P., Bai, T., & Yu, H. (2024). *YOLO-CCA: A Context-Based Approach for Traffic Sign Detection*. arXiv:2412.04289.
- Lin, Z., Wu, Y., Ma, Y., Chen, J., Zhang, R., Wu, J., Yin, G., & Lin, L. (2025). *YOLO-LLTS: Real-Time Low-Light Traffic Sign Detection via Prior-Guided Enhancement and Multi-Branch Feature Interaction*. arXiv:2503.13883.

APPENDIX – MODEL STRUCTURE

	from	n	params	module	arguments
0	-1	1	464	ultralytics.nn.modules.conv.Conv	[3, 16, 3, 2]
1	-1	1	4672	ultralytics.nn.modules.conv.Conv	[16, 32, 3, 2]
2	-1	1	6640	ultralytics.nn.modules.block.C3k2	[32, 64, 1, False, 0.25]
3	-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
4	-1	1	26080	ultralytics.nn.modules.block.C3k2	[64, 128, 1, False, 0.25]
5	-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
6	-1	1	87040	ultralytics.nn.modules.block.C3k2	[128, 128, 1, True]
7	-1	1	295424	ultralytics.nn.modules.conv.Conv	[128, 256, 3, 2]
8	-1	1	346112	ultralytics.nn.modules.block.C3k2	[256, 256, 1, True]
9	-1	1	164608	ultralytics.nn.modules.block.SPPF	[256, 256, 5, 3, True]
10	-1	1	249728	ultralytics.nn.modules.block.C2PSA	[256, 256, 1]
11	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
12	[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat	[1]
13	-1	1	119808	ultralytics.nn.modules.block.C3k2	[384, 128, 1, True]
14	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
15	[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat	[1]
16	-1	1	34304	ultralytics.nn.modules.block.C3k2	[256, 64, 1, True]
17	-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
18	[-1, 13]	1	0	ultralytics.nn.modules.conv.Concat	[1]
19	-1	1	95232	ultralytics.nn.modules.block.C3k2	[192, 128, 1, True]
20	-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
21	[-1, 10]	1	0	ultralytics.nn.modules.conv.Concat	[1]
22	-1	1	463104	ultralytics.nn.modules.block.C3k2	[384, 256, 1, True, 0.5, True]
23	[16, 19, 22]	1	247026	ultralytics.nn.modules.head.Detect	[15, 1, True, [64, 128, 256]]

OL026n summary: 260 layers, 2,509,650 parameters, 2,509,650 gradients, 5.8 GFL0Ps